COURSE HANDBOOK

ELECTRONICS & EMBEDDED SYSTEMS ENGINEERING

Professional Re-entry Program

A practical 20-week refresher for engineers returning after a career break - designed for minimum hardware and maximum transferable engineering skill.



Edition: 30 October 2026 | Free and official learning resources
Study plan: 8-12 hours/week | Prior experience assumed

How to use this handbook

This course is for an electronics or embedded-systems engineer who previously studied or practiced the field and has been away for a year or longer. It is deliberately **not** a general-beginner course and does not assume access to an electronics laboratory. The objective is to rebuild practical judgment, not simply to finish tutorials.

Suggested pace: 20 weeks at 8-12 hours/week, or roughly **160-240 hours**. Weeks are guidance rather than hard deadlines. Use the entry diagnostic to shorten known material and extend any weak module. All project metrics below are **proposed acceptance targets**, not performance guarantees.

> **The core idea: every module has a computer-only path, a small physical extension, written evidence, and a pass condition. Start with simulation and host-based C. Buy hardware only after it clearly enables a measurement or behavior that cannot be validated in software.**

By graduation, the learner should be able to

- Explain a circuit from its schematic; size common components; and justify electrical limits from datasheets.
- Write testable embedded C; use memory and pointers safely; and use debuggers and build tools.
- Read a Cortex-M reference manual; reason about registers, clocks, interrupts, peripherals and DMA.
- Design robust framed communication using UART, I2C, SPI, CAN and RS-485 concepts, and distinguish simulated behavior from verified hardware.
- Build and diagnose a basic multitask RTOS application, a Linux userspace tool and a PCB design.
- Produce professional technical evidence: source code, schematics, calculations, tests, known limitations and a reproducible README.

Contents

How to use this handbook	2
Contents	3
A low-cost learning strategy	4
Entry diagnostic - 2 to 4 hours	4
20-week roadmap	5
01 Electronic fundamentals	6
02 Components and practical circuit design	7
03 Embedded C and software foundations	8
04 ARM Cortex-M and STM32 architecture	9
05 STM32 peripherals and bare-metal timing	10
06 Embedded communication protocols	11
07 Measurement, ADC accuracy and digital filtering	12
08 RTOS concepts and FreeRTOS	14
09 Linux and embedded Linux foundations	15
10 Schematic, PCB and hardware integration	16
11 Debugging, testing, reliability and engineering workflow	17
12 Integrated capstone project	18
Detailed capstone specification	20
Assessment rubric and portfolio standard	21
Choosing a specialization after the refresher	21
Verified resource library	22
Safety and honest validation	23

A low-cost learning strategy

Tier	What is needed	What it really proves
A - No electronics hardware	Existing computer; C toolchain; KiCad/ngspice or LTspice; Linux or Linux VM; version control; Renode where supported; FreeRTOS POSIX port.	Circuit math and simulation; algorithm correctness; build/debug; protocol framing; RTOS concept tests; schematic and PCB checks.
B - One board (recommended)	One NUCLEO-F446RE (or comparable documented STM32), correct USB cable, a few jumper wires; optional LED and resistor.	Real register/clock configuration, interrupts, pin timing, ADC behavior, SWD debugging and basic board integration.
C - Optional measurement tools	Basic multimeter; optionally inexpensive logic analyzer; oscilloscope only if already available/borrowable.	Electrical voltage/current confirmation, waveforms, edges, noise, signal integrity and peripheral electrical behavior.

Do not buy initially: a second microcontroller, CAN analyzer, RS-485 interface, a dedicated FPGA board, oscilloscope, bench supply, motor driver, battery pack, or custom PCB fabrication. These are not required to finish the computer-only track. Hardware-focused engineering proficiency will remain provisional until some real measurement is performed.

Simulation limitations: SPICE depends on model quality; virtual MCUs do not emulate every STM32F446 peripheral; Linux virtual CAN does not prove CAN transceiver signal integrity; host RTOS simulation cannot prove real MCU interrupt latency, jitter or electrical timing. Record these exclusions in each test report.

Recommended software installation order

Stage	Install	Use
Day 1	Git; GCC/Clang; CMake; GDB; code editor	Compile/test ordinary C and publish a repository.
Day 1-2	KiCad with ngspice, or LTspice	Analyze analog circuits without owning components.
Weeks 4-5	STM32CubeIDE or ARM GNU toolchain; STM32CubeF4 docs	Inspect STM32 firmware projects; compile for MCU targets.
Weeks 5-7	Renode with a documented supported board, if usable for the specific exercise	Run selected MCU examples virtually; check peripheral coverage before planning exercises.
Weeks 10-14	FreeRTOS GCC POSIX port; Linux VM / native Linux	Exercise tasks, synchronization, sockets/serial and tools.
Optional	Zephyr SDK with native_sim	Compare another RTOS architecture without adding a board.

Entry diagnostic - 2 to 4 hours

Before beginning, complete the following independently. Keep the results as a baseline. Do not use a score to shame the learner; use it to choose where to invest time.

- **Electronics:** calculate a divider with loading; choose a resistor power rating; predict the step response of an RC network; explain a MOSFET flyback diode.

- **C:** implement a bounded circular buffer; explain a pointer to a struct; write a bit-mask update; identify an out-of-bounds access.
- **STM32:** draw the boot-to-main path; explain a timer overflow; distinguish a peripheral register, an ISR and a DMA transfer.
- **Protocols:** explain UART framing, an I2C ACK, SPI clock polarity/phase and the difference between CAN arbitration and CRC.
- **Tools:** compile a C project using a build system, set a breakpoint in GDB, and commit changes in Git.

Suggested scoring: 0 = cannot explain; 1 = can explain but not implement; 2 = can implement with references; 3 = can implement, test and debug independently. A module with weak results stays on the standard path; strong results can be completed with its exit assignment rather than all guided exercises.

20-week roadmap

Week	Main focus	Module
01	Electrical laws, RC/RL and measurements	M01
02	Components, amplifiers, regulators and protection	M02
03-04	C, pointers, memory, parsing, testing and build tools	M03
05	Cortex-M and STM32 core architecture	M04
06-07	STM32 GPIO, EXTI, timers, ADC, PWM and DMA	M05
08-09	UART, I2C, SPI, CAN, RS-485 and robust protocols	M06
10	Sampling, signal conditioning and calibration	M07
11-12	FreeRTOS tasks, queues, synchronization and timing	M08
13-14	Linux userspace C and device interfaces	M09
15	KiCad schematic, layout, DRC and manufacturability	M10
16	Debugging, fault injection and test design	M11
17-20	System capstone: design, integrate, validate, handoff	M12

Milestones

- **Gate 1 - after week 4:** evidence of circuit calculations, SPICE plots, and an independently tested C parser.
- **Gate 2 - after week 9:** explain STM32 register/interrupt mechanisms; demonstrate or clearly simulate peripheral drivers and communication faults.
- **Gate 3 - after week 16:** RTOS + Linux data flow, PCB design check reports, a systematic debugging log and capstone-ready interfaces.
- **Gate 4 - week 20:** a documented, repeatable integrated system with acceptance test evidence.

01 Electronic fundamentals

WEEK(S) 1 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Rebuild fast electrical intuition: predict a circuit before trusting a simulator or instrument.

Knowledge and engineering decisions

- **Current, voltage, resistance and power.** Use Ohm's law and $P = VI = I^2R$. Understand passive sign convention and why a resistor power rating needs margin.
- **Kirchhoff laws and network reduction.** Solve series/parallel paths, loaded dividers, nodal circuits and simple Thevenin equivalents; identify invalid ideal-source assumptions.
- **Capacitance and inductance.** Explain stored energy, initial conditions and RC/RL transients. Use $\tau = RC$ and $\tau = L/R$ for first-order circuits.
- **AC and frequency response.** Distinguish peak, RMS, average and phase; identify reactance, cutoff frequency and how a first-order filter changes gain.
- **Measurement literacy.** Model multimeter loading, grounding and source impedance; state whether a result is calculation, simulation or observation.

Guided exercises - recommended sequence

01. Solve five circuits on paper: a loaded divider, current limiter, mixed-resistor network, RC transient and RL transient. Include units and assumptions.

02. Build the same circuits in KiCad ngspice or LTspice; run operating-point, transient and AC analyses as applicable.

03. Vary resistor/capacitor values by ±10% in simulation; compare predicted versus simulated responses and explain deviations.

Minimal-hardware learning path

Computer-only: Fully feasible with SPICE and a calculator; export plots and annotate equations.

Optional one-board / physical extension: On an optional board, use a digital output to apply a step to a low-voltage RC network; measure with a multimeter or scope only if available.

Required evidence and exit criteria

Submit: Calculation sheets, SPICE file, three plots, and a one-page note on model assumptions.

Pass when: Correctly calculate current, power and first-order time constants; distinguish meter/simulator limitations.

Common pitfalls: Mistaking simulation accuracy for component tolerance; forgetting source and instrument impedance.

Where to learn - verified resources

[R01] All About Circuits - free electrical engineering textbook - Read the DC, AC and reference volumes; reproduce examples.

[R02] Analog Devices - LTspice download and learning resources - Use the official simulator tutorials and demonstration circuits.

[R04] KiCad schematic editor and ngspice simulation documentation - Alternatively simulate in KiCad without extra software.

02 Components and practical circuit design

WEEK(S) 2 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Learn to select common devices from datasheets and recognize real-world limitations.

Knowledge and engineering decisions

- **Passive devices:** resistor tolerance, temperature and pulse rating; capacitor dielectric, voltage derating and ESR; inductor saturation and ripple-current capability.
- **Protection devices:** rectifier, Schottky, Zener and TVS diodes; placement of flyback paths; fuses, reverse-polarity paths and why ratings must match transients.
- **Switching transistors:** BJT base drive; MOSFET VGS thresholds versus specified RDS(on), gate charge, switching and conduction losses; thermal resistance.
- **Operational amplifiers/comparators:** gain and feedback; offset, bias current, common-mode range, rail-to-rail limitations, bandwidth, noise, stability and hysteresis.
- **Power conversion:** LDO dropout and power loss; introductory buck/boost energy flow; feedback, startup, ripple and protection without claiming full converter-design competency.

Guided exercises - recommended sequence

01. Choose a resistor, diode, capacitor and MOSFET for a hypothetical 3.3 V/5 V switching function; cite each chosen datasheet parameter.

02. Simulate a MOSFET load switch with a flyback diode (use an inductive load), then remove the diode in simulation and inspect voltage stress.

03. Design a comparator threshold with hysteresis and an op-amp signal scaler, using models suitable for the operating voltage.

04. Prepare a short component-selection table: nominal vs worst-case ratings, power margin and substitutes.

Minimal-hardware learning path

Computer-only: SPICE, manufacturer datasheets and a spreadsheet cover the complete required project.

Optional one-board / physical extension: Use LED/resistor, potentiometer and MOSFET at low voltage if available; avoid mains and high-energy inductive loads.

Required evidence and exit criteria

Submit: 2-3 circuit simulations, a component-selection matrix, dissipation calculations and failure-case analysis.

Pass when: Explain why the selected parts work at worst-case limits and identify at least two failure mechanisms.

Common pitfalls: Treating VGS(th) as a fully-on gate specification; treating all 10 µF capacitors as equivalent.

Where to learn - verified resources

[R01] All About Circuits - free electrical engineering textbook - Study semiconductor chapters, device basics and troubleshooting.

[R03] Texas Instruments - Precision Labs: Operational Amplifiers - Follow op-amp precision topics with quizzes and lab problems.

[R02] Analog Devices - LTspice download and learning resources - Simulate the analog and switching circuits.

03 Embedded C and software foundations

WEEK(S) 3-4 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Write predictable, portable C modules that can be tested on a computer before deployment to an MCU.

Knowledge and engineering decisions

- **Types and storage:** integer widths, signedness, fixed-width types, arrays, pointers, alignment, struct packing assumptions, scope and lifetime.
- **Control and abstraction:** functions, function pointers, callbacks, modular APIs, separate compilation, headers, include guards and enum-driven state machines.
- **Bit operations:** set/clear/test masks, shifts, priority of operators, safe register update patterns, and why volatile does not provide atomicity.
- **Memory correctness:** stack versus heap; dangling pointers, bounds checks, endianness, integer overflow, aliasing, error codes and defensive input validation.
- **Development practice:** GCC warnings (-Wall -Wextra -Wconversion where useful), address/undefined-behavior sanitizers on host, CMake, GDB and unit tests.

Guided exercises - recommended sequence

01. Create a CMake-based library with bounded ring buffer APIs: init, push, pop, capacity and overflow behavior.

02. Implement a binary packet parser (preamble, length, message type, payload, CRC or checksum); tolerate byte-by-byte and fragmented input.

03. Test a zero-length frame, truncated frame, maximum-length frame, invalid CRC, buffer wrap-around, and multiple frames.

04. Run GDB on a deliberately broken version; document the error, breakpoint evidence and corrected behavior.

Minimal-hardware learning path

Computer-only: The entire module runs on a normal Windows/Linux/macOS computer with a C compiler and debugger.

Optional one-board / physical extension: Later reuse the same parser library on the STM32; no change to parsing behavior should be needed.

Required evidence and exit criteria

Submit: Repository, CMake target, clean API/header, tests, warning-free normal build, sanitizer results and README.

Pass when: Passes all tests, handles malformed inputs without out-of-bounds accesses, and can explain pointer/ownership decisions.

Common pitfalls: Conflating pointers with arrays; relying on undefined struct layout over a wire; ignoring negative tests.

Where to learn - verified resources

[R06] Beej's Guide to C Programming - Guided C review and exercises, including pointers and memory.

[R05] GNU - C Language Intro and Reference Manual - GNU reference for language semantics and C basics.

[R07] CMake - official hands-on tutorial - CMake step-by-step for libraries and tests.

[R08] GDB - official documentation - GDB debugging reference.

[R09] Unity - embedded C unit test framework - Practical unit testing for embedded C.

04 ARM Cortex-M and STM32 architecture

WEEK(S) 5 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Understand how firmware becomes a running application on Cortex-M and how hardware registers control peripherals.

Knowledge and engineering decisions

- **Architecture:** CPU core, registers, instruction execution, privileged modes, stack pointer, exception model, NVIC and SysTick.
- **Memory system:** vector table, Flash, SRAM, memory-mapped I/O, linker sections, startup code, reset handler and initialization before main().
- **Clock and reset:** oscillator sources, PLL, AHB/APB buses, prescalers, peripheral clock gating, reset domains and watchdog basics.
- **Registers:** access size, address/bit fields, read-modify-write, reserved bits and concurrency risks during ISR and normal-code access.
- **Board vs MCU:** learn to distinguish ST device reference manual, datasheet, board schematic and errata; each answers a different question.

Guided exercises - recommended sequence

01. Draw a startup chain from reset through vector fetch and C runtime initialization to main().

02. From RM0390, identify the RCC, GPIO and timer registers needed to enable a port and toggle a pin; write pseudocode without CubeMX.

03. Inspect a compiled ELF using objdump/readelf and locate .text, .data, .bss, stack and the vector table (when produced).

04. Explain one real erratum or limitation using the manufacturer documentation rather than guessing.

Minimal-hardware learning path

Computer-only: Read ST manuals, inspect an STM32 project, build it for ARM and use disassembly; a supported Renode example can help but is not a drop-in F446 silicon replacement.

Optional one-board / physical extension: If a NUCLEO board is available, flash a blinky build, halt at reset with SWD and inspect RCC/GPIO registers.

Required evidence and exit criteria

Submit: Annotated memory map, startup diagram, compiled project and register-tracing notes.

Pass when: Explain why a peripheral fails when its clock is disabled and trace a program from reset to its first GPIO operation.

> **Common pitfalls:** Confusing the board layout with MCU architecture; using an emulator model as proof of undocumented MCU behavior.

Where to learn - verified resources

[R10] ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata - Primary MCU reference manual, datasheet, programming manual and errata.

[R26] ST - Moving from 8 to 32 bits, startup and register training - Free ST training on registers, startup and low-layer APIs.

[R27] ST - STM32 boot and startup tips - Targeted boot/startup reference.

[R15] Renode - supported boards and STM32F4 Discovery examples - Check Renode supported-board and peripheral coverage.

---

05 STM32 peripherals and bare-metal timing

WEEK(S) 6-7 | LEARNING MODE: theory + hands-on + evidence

> **Module purpose.** Configure and reason about MCU peripherals using reference-manual knowledge, HAL/LL where appropriate, and tests.

Knowledge and engineering decisions

- **GPIO/EXTI:** input/output configurations, alternate functions, pull-up/down, output type, edge-triggered interrupts and debouncing.
- **Timers and PWM:** clock calculation, prescaler, auto-reload, capture/compare, duty cycle, timer-triggered scheduling and overflow.
- **Analog acquisition:** ADC reference, resolution, sample time, channels, timer-triggered conversions and simple scaling.
- **DMA and interrupts:** transfer ownership, circular buffering, ISR latency, half/full-transfer events, interrupt priority and data races.
- **System integrity:** watchdog setup, nonblocking state machines, clock verification, assertions and timeouts rather than uncontrolled busy waits.

Guided exercises - recommended sequence

01. Derive timer register settings for two target frequencies; calculate actual frequency from clock configuration.

02. Create HAL/LL GPIO + EXTI + timer + PWM firmware; keep application logic separate from MCU-specific code.

03. Add timer-triggered ADC sampling with DMA buffer handling and a UART report; use synthetic input data when hardware is absent.

04. Inject a missing event or stalled operation; verify a bounded timeout and document the recovery path.

Minimal-hardware learning path

Computer-only: Write peripheral abstraction interfaces and test timing math on the host. Compile the STM32 target and use Renode only for peripherals known to be modeled; mark unmeasurable functions as unverified.

Optional one-board / physical extension: One STM32 Nucleo board plus its onboard LED/button suffices for GPIO/EXTI/timer and SWD; a pot/resistor is optional for analog acquisition.

Required evidence and exit criteria

Submit: Firmware with a board-support layer, clock diagram, timing calculations, logging and test matrix.

Pass when: Demonstrate a nonblocking program, explain DMA data ownership and measure at least one real timing property if hardware exists.

> **Common pitfalls:** Treating HAL-generated defaults as unexplained truth; performing extensive work inside interrupts; blocking forever on a peripheral.

Where to learn - verified resources

[R10] ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata - Read the RCC, GPIO, timer, ADC, DMA and NVIC-related chapters.

[R14] ST - STM32CubeIDE resources - Use ST development environment and debugging documentation.

[R25] ST - STM32CubeIDE basics course - Follow GPIO, EXTI, PWM, ADC/DMA and USART labs; adapt them to your own board.

[R13] ST - STM32 online training - Supplement with free peripheral training.

---

06 Embedded communication protocols

WEEK(S) 8-9 | LEARNING MODE: theory + hands-on + evidence

> **Module purpose.** Select, implement and debug realistic data links while separating logical protocol tests from electrical bus validation.

Knowledge and engineering decisions

- **UART:** start/data/stop/parity bits, baud-rate error, framing, receive buffers, DMA/IDLE strategy, overflow and resynchronization.
- **I2C:** open-drain lines and pull-ups, device addressing, ACK/NACK, clock stretching, arbitration and bus recovery concepts.
- **SPI:** master/slave behavior, clock phase/polarity, chip-select timing, full-duplex transfers and command-response framing.
- **CAN:** message identifiers, arbitration, bit timing and acceptance filters, CRC/acknowledgment/error states; physical transceiver and termination must be validated separately.
- **RS-485 and Modbus RTU:** transceiver enable, differential line termination/biasing, half-duplex collisions, framing, CRC and silent interval requirements.

Guided exercises - recommended sequence

01. Extend the C parser from Module 3 to carry sequence number, message ID, payload length, checksum, timeout and resynchronization.

02. Test I2C and SPI exchanges with mocked peripheral calls, including a missing ACK, wrong device response and timeouts.

03. On native Linux, use a virtual CAN interface (vcan) where kernel support is available to send, receive and filter CAN frames.

04. Write a short protocol comparison explaining which interface best fits a nearby sensor, EEPROM, PC connection and distributed network.

Minimal-hardware learning path

Computer-only: Mock UART/I2C/SPI responses and use Linux vcan for protocol-level CAN; no CAN transceiver or second MCU is mandatory.

Optional one-board / physical extension: Use the STM32 serial interface and onboard ST-LINK virtual COM port where supported; physical CAN/RS-485 tests require transceiver hardware and are deferred.

Required evidence and exit criteria

Submit: Protocol specification, annotated frame format, test vectors, fault cases and Linux/MCU logs as available.

Pass when: Reject bad frames safely, recover synchronization, distinguish physical faults from software faults and explain bus-level design trade-offs.

> **Common pitfalls:** Calling Linux vcan a hardware CAN test; assuming I2C needs no pull-ups; ignoring I/O voltage compatibility.

Where to learn - verified resources

[R10] ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata - STM32 UART, I2C, SPI and CAN peripheral chapters.

[R19] Linux kernel - SocketCAN and virtual CAN documentation - Official Linux SocketCAN and vcan documentation.

[R20] Linux manual pages - termios - Linux serial port configuration reference.

[R12] ST - free STM32 MOOCs - ST free communication-peripheral training.

---

07 Measurement, ADC accuracy and digital filtering

WEEK(S) 10 | LEARNING MODE: theory + hands-on + evidence

> **Module purpose.** Understand the complete measurement chain from sensor to numerical result, including limitations and uncertainty.

Knowledge and engineering decisions

- **Signal chain:** sensor, excitation/reference, front-end scaling, anti-alias filter, ADC and engineering-unit conversion.
- **Sampling:** sample rate, Nyquist condition, aliasing, resolution, acquisition time, sample-and-hold source impedance and DMA data flow.
- **Errors:** quantization, offset, gain error, reference drift, noise, input loading and temperature effects. Distinguish precision from absolute accuracy.
- **Filtering:** moving average, first-order IIR/exponential filtering and median filtering; compare noise reduction and added latency.
- **Calibration:** one/two-point correction, test fixtures, documentation of units, rounding and uncertainty budget.

Guided exercises - recommended sequence

01. Create synthetic voltage or temperature data with configurable Gaussian-like random noise, bias, drift and spikes.

02. Implement moving average and first-order IIR filters in host C; export raw and filtered values and analyze step-response delay.

03. Calculate ADC LSB size for an assumed Vref and bit depth; derive a conversion formula and a simple worst-case error budget.

04. Optional: acquire a low-voltage ADC signal from a potentiometer and compare to an external multimeter, stating its limitations.

Minimal-hardware learning path

Computer-only: Entire analysis can be performed with simulated sensor vectors, C algorithms and spreadsheets.

Optional one-board / physical extension: One STM32 ADC input and a potentiometer/resistor divider are sufficient for a practical extension; no precision instrument is assumed.

Required evidence and exit criteria

Submit: Filter implementation, plotted raw/filtered data, calibration table and measurement uncertainty notes.

Pass when: Explain why more averaging does not remove bias or reference error, and choose sample rate and filter to meet a stated response target.

> **Common pitfalls:** Using ADC bit count as a statement of true accuracy; filtering noise without accounting for latency.

Where to learn - verified resources

[R16] ST - STM32F446RE product page (including AN2834 ADC accuracy) - Find AN2834 via the official product page for ADC accuracy guidance.

[R03] Texas Instruments - Precision Labs: Operational Amplifiers - TI signal-chain lessons on noise, offset and limitations.

[R04] KiCad schematic editor and ngspice simulation documentation - Model the analog input filter using ngspice.

08 RTOS concepts and FreeRTOS

WEEK(S) 11-12 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Build correct concurrent systems and understand the boundary between task-level and interrupt-level work.

Knowledge and engineering decisions

- **Scheduling:** periodic versus event-driven tasks, preemption, priorities, delays, deadlines, jitter and response-time measurement.
- **Coordination:** queues, semaphores, mutexes, direct task notifications, event groups and their differing ownership/notification semantics.
- **Failure modes:** deadlocks, starvation, priority inversion, race conditions, lost wakeups and stack overflow detection.
- **Memory/time:** static/dynamic task allocation choices, bounded queues, task stack sizing, watchdog and health monitoring.
- **Architecture:** separate data acquisition from computation, communication and logging; use bounded messages, ownership and fault paths.

Guided exercises - recommended sequence

01. Use the supported FreeRTOS GCC POSIX simulator port to create producer, consumer and supervisor tasks.

02. Implement bounded queue behavior and instrumentation for queue full/empty events, processing latency and missed events.

03. Deliberately create a race or inversion scenario, explain why it happens, then correct it with appropriate synchronization.

04. Port the task topology to STM32 if hardware exists, and compare measured behavior with host simulation assumptions.

Minimal-hardware learning path

Computer-only: FreeRTOS includes a GCC POSIX simulator port; begin on Linux without any MCU board. Host timing is not representative of microcontroller real-time guarantees.

Optional one-board / physical extension: Optional STM32 Nucleo deployment validates stack sizing, interrupt priorities and actual tick/ISR interactions.

Required evidence and exit criteria

Submit: Task diagram, source, test logs, stack/queue budget and concurrency fault analysis.

Pass when: Explain why each primitive was chosen, show queue-overflow handling and distinguish functional correctness from timing determinism.

Common pitfalls: Using a mutex in ISR context; assuming volatile fixes concurrency; blocking higher-priority work indefinitely.

Where to learn - verified resources

[R17] FreeRTOS - official kernel documentation - Official FreeRTOS task, queue, semaphore, timer and notification documentation.

[R18] FreeRTOS-Kernel sources, including GCC POSIX simulator port - POSIX simulator port and kernel sources.

[R12] ST - free STM32 MOOCs - ST FreeRTOS-on-STM32 MOOC for MCU specifics.

09 Linux and embedded Linux foundations

WEEK(S) 13-14 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Understand Linux as a development host and as a possible embedded system, without requiring a Raspberry Pi or custom board.

Knowledge and engineering decisions

- **Development and OS model:** shell, processes, virtual memory, ELF, system calls, files, permissions, toolchain and dynamic libraries.
- **POSIX programming:** file descriptors, open/read/write, errno, poll, signals, fork/exec/wait and thread synchronization.
- **Device I/O:** terminals and termios; pseudo-terminals; sockets; Linux CAN/vcan; GPIO character-device API as a conceptual example.
- **Embedded Linux picture:** bootloader, kernel, device tree, root filesystem, init services, cross-toolchains and driver/userspace responsibility.
- **Debug/operations:** logs, strace concept, GDB, service recovery, startup scripts, deployment and repeatable build procedures.

Guided exercises - recommended sequence

01. Create a PTY pair or use a PTY test harness; write a Linux C program that speaks the framed protocol from Module 3.

02. Use poll to receive partial frames without blocking all application work; test disconnect/reconnect, timeouts and Ctrl-C cleanup.

03. Create sender/receiver threads or processes; add log rotation or capped storage and meaningful exit codes.

04. Optional: use Zephyr native_sim to compare a different embedded software execution model, noting its simulated hardware scope.

Minimal-hardware learning path

Computer-only: Use Linux native/VM. WSL2 is sufficient for many userspace exercises, but Linux kernel modules, GPIO hardware and some device passthrough scenarios require native Linux or a suitable VM.

Optional one-board / physical extension: Optional single STM32 board can be the live serial endpoint; otherwise the PTY harness is the endpoint.

Required evidence and exit criteria

Submit: Linux daemon/CLI, protocol tests, reproducible build, log files and a short Linux boot architecture diagram.

Pass when: Gracefully handle partial reads and disconnects, demonstrate correct cleanup, and explain when a kernel driver is inappropriate.

Common pitfalls: Assuming a read() returns a complete frame; equating WSL with direct hardware access; choosing kernel code when userspace suffices.

Where to learn - verified resources

[R20] Linux manual pages - termios - Consult termios reference for raw serial configuration.

[R19] Linux kernel - SocketCAN and virtual CAN documentation - Virtual CAN interface and socket API.

[R23] Linux kernel - GPIO character device userspace API - Kernel documentation on modern userspace GPIO API.

[R21] Zephyr - native_sim board documentation - Zephyr native_sim board, optional.

[R22] Zephyr - official getting-started guide - Official Zephyr setup for the optional extension.

10 Schematic, PCB and hardware integration

WEEK(S) 15 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Produce a reviewable electronic design, even if the board is never manufactured.

Knowledge and engineering decisions

- **Schematic quality:** block diagrams, clear nets, power rails, pin names, decoupling, bypassing, protection, connectors and test points.
- **Component selection:** footprint/pad verification, absolute maximum limits, operating ranges, tolerances, availability and lifecycle.
- **PCB layout:** placement from current paths, return-current continuity, power/ground planes, analog/digital partition, trace current and thermal issues.
- **Integrity:** decoupling proximity, regulator loop area, switching-node handling, ESD strategy, noise coupling and basic EMC awareness.
- **Review:** ERC and DRC, 3D footprint check, fabrication layers, BOM consistency, polarity and pin-one verification.

Guided exercises - recommended sequence

01. Design a small 3.3 V sensor interface board with power input, protection, microcontroller connector, I2C sensor header and one analog input filter.

02. Create a spreadsheet matching symbols, manufacturer pinout and footprints; review every connector orientation.

03. Complete layout using a realistic manufacturer ruleset; run ERC and DRC, document and resolve every report.

04. Export Gerber/drill/BOM, assemble a digital design review pack, and identify two likely EMI/ESD weaknesses.

Minimal-hardware learning path

Computer-only: KiCad does not require purchased hardware or PCB fabrication. Simulation, DRC and manufacturing file reviews are sufficient for the design-only deliverable.

Optional one-board / physical extension: Optional fabrication is appropriate only if a mentor or lab can support basic board bring-up and measurements.

Required evidence and exit criteria

Submit: Schematic, PCB file, 3D images, ERC/DRC logs, BOM and design-review checklist.

Pass when: Explain every major placement/grounding decision and pass rule checks without hiding unresolved problems.

Common pitfalls: Assuming ERC/DRC verifies function; choosing footprints from names only; sharing noisy return paths indiscriminately.

Where to learn - verified resources

[R04] KiCad schematic editor and ngspice simulation documentation - Read KiCad schematic and simulation documentation.

[R24] KiCad - PCB editor and design-rule checker documentation - Read the official PCB editor, DRC and fabrication instructions.

[R03] Texas Instruments - Precision Labs: Operational Amplifiers - Use TI PCB-layout analog lessons for grounding and bypassing.

11 Debugging, testing, reliability and engineering workflow

WEEK(S) 16 | LEARNING MODE: theory + hands-on + evidence

Module purpose. Learn the discipline of making fault diagnoses based on reproducible evidence rather than guesses.

Knowledge and engineering decisions

- **Debugging tools:** GDB, breakpoints, watchpoints, memory inspection, stack/backtrace, logging levels and minimal reproduction.
- **Firmware failure analysis:** HardFault state, invalid access, interrupts, race conditions, stack overrun, watchdog resets and misconfigured clocks.
- **Test strategy:** unit, integration, system and endurance tests; positive/negative inputs; boundary-value cases; fault injection.
- **Timing and reliability:** queue/packet loss rates, latencies, error counters, graceful degradation and recovery paths.
- **Engineering workflow:** Git branches/commits, code review checklist, requirement-to-test traceability and evidence-based defect reports.

Guided exercises - recommended sequence

01. Introduce three faults into prior modules: a buffer boundary bug, invalid packet and blocked/missed RTOS message.

02. For each, record symptom, hypothesis, experiment, evidence, root cause, fix and regression test.

03. Enable host compiler diagnostics and sanitizers; run repeatable test harnesses and compare regression results.

04. Write an acceptance matrix showing requirement, test method, observed result, caveat and pass/fail status.

Minimal-hardware learning path

Computer-only: All software defects can be investigated using a normal PC, GDB, tests and simulated inputs.

Optional one-board / physical extension: If a board exists, add SWD breakpoint/watchpoint tests and one watchdog/reset fault-injection trial.

Required evidence and exit criteria

Submit: Three fault reports, test matrix, CI-ready local test script and final design review notes.

Pass when: Provide a root-cause explanation and a test that would catch the bug if it reappears.

> **Common pitfalls:** Changing several variables simultaneously during diagnosis; presenting unsupported timing figures as measured.

Where to learn - verified resources

[R08] GDB - official documentation - Use the official GDB manual.

[R09] Unity - embedded C unit test framework - Use Unity or comparable tests for C.

[R07] CMake - official hands-on tutorial - Use CMake/CTest for repeatable builds and tests.

[R10] ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata - Read STM32 errata when a hardware-specific failure is suspected.

---

12 Integrated capstone project

WEEK(S) 17-20 | LEARNING MODE: theory + hands-on + evidence

> **Module purpose.** Combine the previous work into a small, credible, reproducible system that demonstrates engineering judgment.

Knowledge and engineering decisions

- **System requirement:** define what the device acquires, how it communicates, which faults are handled and what constitutes passing.
- **Architecture:** separate measurement drivers, processing, protocol parsing, communication, persistence and host integration.
- **Portability:** keep application/protocol logic host-testable with hardware adapter interfaces; compile and test both host and target layers where feasible.
- **Validation:** schedule and log tests, fault injection, stress/soak testing, resource budgets, design review and final limitations.
- **Handoff:** make a new engineer able to clone, build, simulate, inspect evidence and understand unresolved risks.

Guided exercises - recommended sequence

01. Week 17: write one-page specification, block diagram, interface contract, planned tests and realistic timing assumptions.

02. Week 18: implement host-side data source, application state machine, protocol library, logging and test harness.

03. Week 19: integrate FreeRTOS or another selected RTOS path where feasible, plus optional STM32 firmware and a KiCad interface schematic.

04. Week 20: execute acceptance and fault-injection tests; write reproducible build instructions, known limitations and final engineering report.

Minimal-hardware learning path

Computer-only: Complete a virtual diagnostic/data-acquisition controller: simulated sensor values -> C data-processing library -> framed protocol -> Linux logging/CLI. Include STM32 build artifacts and virtual peripheral validation only where supported.

Optional one-board / physical extension: With one STM32 board, replace the simulated producer with MCU data capture and stream over USB/UART. An external transceiver or sensor is not essential.

Required evidence and exit criteria

Submit: Working repo, architecture, interface specification, test evidence, PCB files, acceptance report, risk register and demonstration recording or transcript.

Pass when: Another engineer can build and run it from the README; error recovery is demonstrated; claims match what was actually tested.

> **Common pitfalls:** Over-scoping before validating one end-to-end data path; claiming real-time or electrical compliance from simulation alone.

Where to learn - verified resources

[R10] ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata - STM32 manuals for implementation details and limitations.

[R17] FreeRTOS - official kernel documentation - RTOS kernel API reference.

[R19] Linux kernel - SocketCAN and virtual CAN documentation - Optional virtual CAN extension.

[R04] KiCad schematic editor and ngspice simulation documentation - KiCad schematics and simulation.

[R08] GDB - official documentation - GDB debug and test evidence.

Detailed capstone specification

Default project: a compact diagnostic and data-acquisition controller. It can be built as host C applications plus virtual devices and later migrated to STM32. This is deliberately feasible with one computer and one optional Nucleo board.

Subsystem	Minimum implementation	Low-hardware acceptance evidence
Virtual / real acquisition	Generate voltage/temperature values on host; optional MCU ADC acquisition.	Known input vectors and timestamps; optional real ADC log.
Firmware/application core	State machine for INIT, RUN, FAULT, RECOVER; bounded queues.	Unit tests and state-transition traces.
Frame transport	Preamble, version, type, length, sequence, payload, checksum/CRC.	Fragmentation, corruption, timeout, overflow and recovery tests.
Linux host	CLI to start/stop/inspect; bounded log file; summaries.	Tested CLI operations and durable CSV/JSON logs.
Hardware design	KiCad power/interface schematic, optionally a PCB layout.	ERC/DRC and documented manual review; no fabrication required.
Verification	Track test outcomes and reproducible procedures.	Named acceptance tests, raw logs, plots and limitation notes.

Proposed numeric targets - verify, do not assume

- **Nominal sample-event schedule:** 100 events per second; a 30-minute simulation expects 100 x 60 x 30 = **180,000 generated sample events**. Demonstrate receipt counts, and account for every intentionally dropped event.
- **Protocol robustness:** every malformed packet is rejected without a crash; recovery is verified using deliberately split and corrupted streams.
- **Persistence:** a 60-minute logging run remains readable and has an explicit disk-space or record-count limit.
- **Fault handling:** simulate sensor disconnection, source pause, queue overflow and host reconnect. State which are auto-recoverable.
- **Timing caveat:** do not claim MCU hard-real-time jitter or analog accuracy based only on a PC or simulator.

Capstone design review checklist

- System architecture matches the specification and identifies all tested versus simulated interfaces.
- Every buffer is bounded; packet and integer lengths are validated before indexing or copying.
- All electrical values have stated units, ranges, error handling and safe low-voltage assumptions.
- The build/test process runs from a fresh clone, with clear toolchain versions and dependencies.
- The acceptance report includes failures, incomplete items, measured results and decisions for next iteration.

Assessment rubric and portfolio standard

Dimension	Weight	Evidence needed
Conceptual explanation	20%	Explains operating principles, limits and design tradeoffs.
Correct implementation	25%	C code, simulation, MCU firmware or schematic that does what it claims.
Tests and debugging	25%	Negative cases, trace evidence, repeatability, measurement limits.
Engineering decisions	15%	Calculations, component ratings, assumptions, risk treatment.
Documentation and handoff	15%	README, source organization, BOM/interface notes, known limitations.

Suggested pass rule: 70/100 overall with no critical safety or correctness defect, plus a successful technical walkthrough. A score is a course-design convention, not an external professional certification. Strong performance also requires the learner to reproduce the build/test without direct instructor guidance.

Repository structure

```
reentry-program/
  README.md
  requirements-and-assumptions.md
  modules/module-01 ... module-11/
  shared/libprotocol/
  capstone/firmware/
  capstone/linux-host/
  capstone/hardware-kicad/
  capstone/tests/
  capstone/evidence/
  capstone/acceptance-report.md
```

Choosing a specialization after the refresher

Target role	Extend next with	Suggested follow-up project
Embedded firmware	Advanced DMA, low power, bootloaders, memory/interrupt fault analysis.	Robust data logger with firmware update and power-state tests.
Embedded Linux	Buildroot, device trees, kernel driver basics, cross-compilation, networking.	Linux SBC to MCU gateway with service supervision.
Electronics / PCB	Power supplies, EMC, signal integrity, analog measurement, thermal design.	Fabricate and validate a protected low-voltage sensor board.
Industrial communications	CAN FD, Modbus, system diagnostics, interoperability, isolated transceivers.	Multi-node gateway with reproducible fault injection.
Real-time systems	Scheduling analysis, RTOS instrumentation, hard deadlines and measurement.	Timed motor/sensor controller with recorded worst-case latency.

Verified resource library

The links below were checked against publisher, manufacturer or official project pages on 30 October 2026. Course authors should re-check tool versions and board-specific prerequisites when the learner begins. Resources are external and may change. Several ST MOOCs demonstrate boards different from the suggested NUCLEO-F446RE; transfer the concepts, not their pin or project configuration verbatim.

R01 All About Circuits - free electrical engineering textbook

https://www.allaboutcircuits.com/textbook/

R02 Analog Devices - LTspice download and learning resources

https://ltspice.analog.com/

R03 Texas Instruments - Precision Labs: Operational Amplifiers

https://www.ti.com/video/series/precision-labs/ti-precision-labs-op-amps.html

R04 KiCad schematic editor and ngspice simulation documentation

https://docs.kicad.org/9.0/en/eeschema/eeschema.html

R05 GNU - C Language Intro and Reference Manual

https://www.gnu.org/software/c-intro-and-ref/manual/

R06 Beej's Guide to C Programming

https://beej.us/guide/bgc/

R07 CMake - official hands-on tutorial

https://cmake.org/cmake/help/latest/guide/tutorial/

R08 GDB - official documentation

https://sourceware.org/gdb/documentation/

R09 Unity - embedded C unit test framework

https://github.com/ThrowTheSwitch/Unity

R10 ST - STM32F446 documentation: RM0390, PM0214, datasheet, errata

https://www.st.com/en/microcontrollers-microprocessors/stm32f446/documentation.html

R11 ST - NUCLEO-F446RE board information

https://www.st.com/en/evaluation-tools/nucleo-f446re.html

R12 ST - free STM32 MOOCs

https://www.st.com/content/st_com/en/support/learning/stm32-moocs.html

R13 ST - STM32 online training

https://www.st.com/content/st_com/en/support/learning/stm32-online-training.html

R14 ST - STM32CubeIDE resources

https://www.st.com/content/st_com/en/stm32cubeide.html

R15 Renode - supported boards and STM32F4 Discovery examples

https://renode.readthedocs.io/en/latest/introduction/supported-boards.html

R16 ST - STM32F446RE product page (including AN2834 ADC accuracy)

https://www.st.com/en/microcontrollers-microprocessors/stm32f446re.html

R17 FreeRTOS - official kernel documentation

https://developer.freertos.org/Documentation/00-Overview

R18 FreeRTOS-Kernel sources, including GCC POSIX simulator port

https://github.com/FreeRTOS/FreeRTOS-Kernel

R19 Linux kernel - SocketCAN and virtual CAN documentation

https://docs.kernel.org/networking/can.html

R20 Linux manual pages - termios

https://man7.org/linux/man-pages/man7/termios.7.html

R21 Zephyr - native_sim board documentation

https://docs.zephyrproject.org/latest/boards/native/native_sim/doc/index.html

R22 Zephyr - official getting-started guide

https://docs.zephyrproject.org/latest/develop/getting_started/

R23 Linux kernel - GPIO character device userspace API

https://docs.kernel.org/userspace-api/gpio/chardev.html

R24 KiCad - PCB editor and design-rule checker documentation

https://docs.kicad.org/9.0/en/pcbnew/pcbnew.html

R25 ST - STM32CubeIDE basics course

https://www.st.com/content/st_com/en/support/learning/stm32-moocs/STM32CubeIDE_basics_MOOC.html

R26 ST - Moving from 8 to 32 bits, startup and register training

https://www.st.com/content/st_com/en/support/learning/stm32-moocs/Moving_from_8_to_32_bits_workshop_MOOC.html

R27 ST - STM32 boot and startup tips

https://www.st.com/content/st_com/en/support/learning/stm32-moocs/STM32_boot_startup_MOOC.html

R28 GNU/Linux - kernel overview and documentation

https://docs.kernel.org/

Safety and honest validation

- All recommended optional experiments are intended for low-voltage, current-limited circuits. Do not use mains-powered, high-voltage or high-energy battery equipment without qualified supervision.
- Never directly connect incompatible voltage domains to a microcontroller pin. Read absolute maximum ratings and recommended operating conditions separately.
- Before powering an assembled board, inspect polarity, pin 1, shorts, expected supply current and regulator output settings.
- An EDA simulation, a passing ERC/DRC, a test run on a PC, and a measured physical signal are different levels of evidence. Label them accordingly.
- Formal EMC, electrical safety, functional safety, environmental qualification and product certifications are outside the scope of this refresher.

Document design: a recommended educational syllabus, not a manufacturer-certified training program. References identify where to learn the underlying techniques; course sequence, pacing, exercises and pass thresholds are original curriculum recommendations.